arXiv:2607.22634v1 [cs.AI] 20 Jun 2026

PRESTO: Prefix-Aligned Tree Drafting for Diffusion Speculative Decoding

Zheng Wang[1,*] **Zhifan Ye**[2,3,*] **Qi Cheng**[3] **Yonggan Fu**[3] **Ziyan Wang**[2] **Feng Zhu**[2]
Haozhe Zhao[1] **Jan Kautz**[3] **Pavlo Molchanov**[3] **Humphrey Shi**[2,3] **Minjia Zhang**[1]

[1]University of Illinois Urbana-Champaign [2]Georgia Institute of Technology [3]NVIDIA
[*]Equal contribution

Abstract: Diffusion Large Language Models (dLLMs) have recently emerged as a promising alternative to autoregressive (AR) LLMs, offering parallel token generation. Recent works have shown that dLLMs are particularly well-suited as draft models for speculative decoding (SD), as they can efficiently generate an entire block of draft tokens in parallel within a single forward pass. However, existing diffusion-based drafting methods primarily rely on linear drafting (single-path), despite diffusion models simultaneously producing multiple candidate tokens at multiple positions that naturally induce a large combinatorial space of possible decoding paths. As a result, these methods can only explore a tiny fraction of the available candidate space, inherently limiting the achievable acceptance length and decoding efficiency. To fully utilize this rich multi-candidate structure, we apply tree-based drafting to diffusion drafters, enabling the exploration of diverse candidate paths. Nevertheless, we find that applying naive tree-based drafting is suboptimal due to a fundamental mismatch between diffusion draft confidence and prefix-based AR verification: diffusion marginals are inherently prefix-blind, which can lead to unreliable path ranking. We propose PRESTO, a principled framework that extends tree-based drafting to diffusion drafters while resolving the fundamental mismatch between diffusion draft confidence and prefix-based AR verification through ***PRE****fix-aligned* ***S****coring* and *priority-based* ***T****ree search* for diffusion speculative dec**O**ding. The key principles behind PRESTO are that (1) candidate ranking should align with the prefix-based nature of AR verification, and (2) tree construction should prioritize candidate paths with high verification potential to maximize acceptance length. We also demonstrate that PRESTO is designed to be a general tree drafting framework applicable to both dedicated diffusion drafter SD and self-speculative dLLMs. Extensive experiments show that PRESTO achieves an average of **1.5**× end-to-end throughput speedup on the state-of-the-art dedicated diffusion drafter SD and an average of **1.12**× on self-speculative diffusion LLMs across diverse benchmarks.

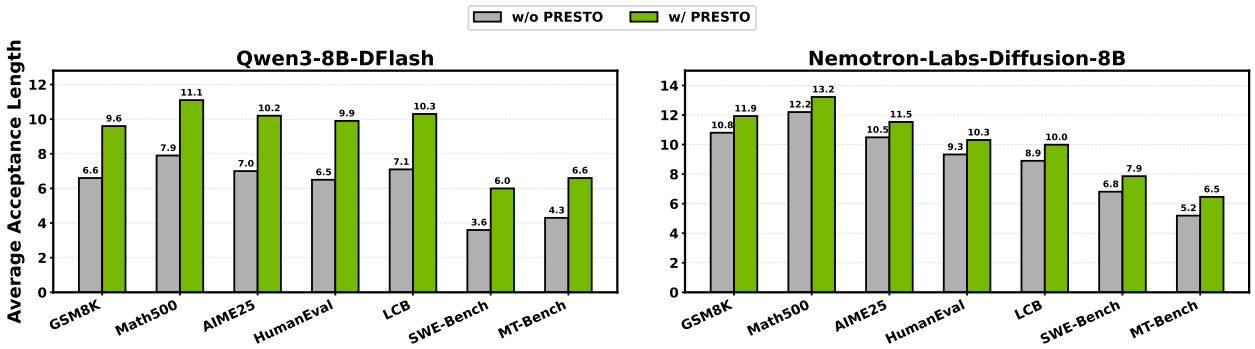


Figure 1 | Acceptance length comparison of PRESTO on diffusion-based SD, including dedicated diffusion drafter SD (Qwen3-8B-DFlash) and self-speculative dLLM (Nemotron-Labs-Diffusion-8B; marginal-only scoring with no prefix-signal) across diverse benchmarks. PRESTO consistently improves acceptance length over the baseline across all tasks, enabling more tokens to be accepted in each verification round.

1. Introduction

Diffusion Large Language Models (dLLMs) have recently emerged as a promising alternative to autoregressive (AR) LLMs, introducing a new paradigm for multi-token generation [1, 2, 3, 4, 5, 6]. However, achieving high-quality generation in practice typically requires iterative denoising procedures and carefully designed token

unmasking strategies, which introduce additional computation and partially offset the efficiency gains [3, 7]. Interestingly, recent works have shown that diffusion models are particularly well-suited as draft models in speculative decoding [8, 9, 10, 11]. Their ability to generate multiple candidate tokens in parallel naturally aligns with the proposal stage of speculative decoding (SD), enabling highly efficient draft token generation. This insight has led to a growing line of work that leverages diffusion models as effective drafters, including dedicated diffusion-based drafter for AR LLMs (e.g., dFlash [10]) and self-speculative dLLMs (e.g., TiDAR [8], I-DLM [11], Nemotron-Labs-Diffusion [12]). For example, dFlash demonstrates that well trained lightweight diffusion drafters can substantially accelerate speculative decoding through efficient parallel draft token generation while maintaining high draft quality, highlighting the practical potential of diffusion-based drafting.

However, existing diffusion-based drafting methods primarily rely on *linear (single-path) drafting*, where tokens are generated along a single trajectory and verified sequentially. While this design is effective in AR settings, it is inherently suboptimal for diffusion models. Due to the non-autoregressive nature of diffusion generation, multiple plausible candidates can exist simultaneously, while the decoding path with the highest acceptance length may not correspond to the top-1 marginal trajectory. Empirically, we observe that even strong diffusion drafters in dFlash yield only modest accepted lengths under single-path drafting (See Figure 2(a)), indicating that a significant portion of the proposal space remains unexplored.

A natural direction to address this limitation is to move beyond linear drafting and explore multiple candidate paths jointly. In AR settings, prior work has shown that introducing tree-based drafting can substantially increase acceptance length by expanding multiple candidate paths in parallel, thereby improving the utilization of each verification step [13, 14, 15, 16, 17, 18]. In particular, since the target model is often not fully compute-bound during verification [8] for small batch size cases, allocating additional drafting compute to explore a richer candidate set can lead to more tokens being accepted per step. The success of tree-based drafting in AR settings raises a key research question:

Can we leverage tree-based drafting in diffusion drafters to further improve the end-to-end throughput of diffusion-based speculative decoding by increasing acceptance length?

While this direction is promising, we find that naively applying tree-based drafting to diffusion models is often suboptimal. Diffusion drafters generate position-wise token distributions in parallel, without strict prefix dependencies, which enables efficient and flexible construction of tree-structured candidates. However, this same property also implies that the resulting scores are marginal token probabilities, rather than prefix-conditioned likelihoods. Consequently, these scores cannot accurately reflect how earlier token choices influence downstream acceptance (e.g., a token that is likely in isolation may become unlikely once conditioned on the chosen prefix), creating a fundamental mismatch between draft scoring and prefix-based AR verification.

In this work, we propose PRESTO, a principled tree-based diffusion drafting framework via *prefix-aligned scoring* and *priority-based tree search*. The key principles behind PRESTO are that (1) candidate ranking should align with the prefix-based nature of AR verification, and (2) tree construction should prioritize candidate paths with high verification potential to maximize acceptance length. Specifically, we minimally adjust the diffusion marginal by incorporating a prefix-dependent correction, so that the resulting score better reflects the acceptance behavior under sequential verification. This score is then used to guide a priority-based expansion strategy, which allocates computation to high-quality candidate paths during tree construction to maximize acceptance length. As a result, PRESTO enables effective tree-based diffusion drafting and significantly improves acceptance length and throughput on both dedicated diffusion-based drafters for AR LLMs and self-speculative dLLMs. Specifically, we summarize our contributions as follows:

- **We identify a fundamental mismatch between diffusion-based draft scoring and prefix-based AR verification.** We show that while diffusion probabilities provide well-calibrated marginal acceptance signals, they do not fully capture the prefix-conditioned nature of AR verification, which can lead to suboptimal path ranking during tree construction and ultimately limit acceptance length.
- **We design prefix-aligned scoring to address the above mismatch.** We formulate a principled, minimal correction to diffusion marginals by incorporating prefix-dependent signals, yielding a scoring function that is both faithful to the diffusion drafters and compatible with prefix-based verification.
- **We formalize and enable tree-based speculative drafting for diffusion drafters.** Building upon the proposed prefix-aligned scoring, we introduce PRESTO, a principled tree-based diffusion drafting

framework that efficiently explores high-quality draft paths through priority-based tree expansion, substantially improving acceptance length and decoding throughput.

- **PRESTO generalizes across both dedicated and self-speculative diffusion SD frameworks.** We further extend PRESTO to self-speculative dLLMs as a drop-in replacement for linear drafting in the parallelized verify-and-draft pipeline, making PRESTO the first tree-based drafting framework applicable to both dedicated diffusion drafters and self-speculative diffusion LLMs. Across diverse benchmarks and model sizes, PRESTO achieves an average of **1.5**× throughput speedup on dFlash and an average of **1.12**× throughput speedup on Nemotron-Labs-Diffusion over linear drafting baselines.

2. Preliminaries

2.1. Tree-based Speculative Decoding

Speculative decoding accelerates autoregressive generation of a target LLM p_T by leveraging a lightweight draft model. To improve acceptance, the draft model can construct a token tree encoding multiple candidate sequences [13, 19, 18]. The target LLM verifies candidates in a prefix-based manner: starting from the root, it accepts tokens sequentially until a mismatch occurs.

Acceptance factorization. For a candidate path $P = (x_1, \ldots, x_k)$, let $a_i \in \{0, 1\}$ denote the indicator that the i-th token is accepted. Acceptance proceeds left-to-right, so by the chain rule

$$\Pr(P \text{ accepted}) = \prod_{i=1}^{k} \Pr\big(a_i = 1 \;\big|\; a_{<i} = 1,\, x_{\le i}\big), \tag{1}$$

where $a_{<i} = 1$ denotes the event that all earlier tokens have been accepted. Each factor depends on the realized prefix $x_{<i}$, making verification intrinsically prefix-conditional.

Tree construction objective. Let $\mathcal{B}_B = \{\mathcal{T} : |\mathcal{T}| \le B\}$ denote token trees with at most B nodes where $\mathcal{T}$ is the tree budget, and let $\alpha_{\mathcal{T}}(P)$ denote the longest prefix of P contained in $\mathcal{T}$. The ideal objective is

$$\mathcal{T}^* \in \arg\max_{\mathcal{T} \in \mathcal{B}_B} \mathbb{E}_{P \sim p_T}[\alpha_{\mathcal{T}}(P)]. \tag{2}$$

Direct optimization is infeasible, as evaluating p_T requires target-model forward passes. We use a surrogate distribution $\tilde{p}$ over candidate paths. A standard decomposition (Appendix A) yields

$$\mathbb{E}_{P \sim \tilde{p}}[\alpha_{\mathcal{T}}(P)] = \sum_{u \in \mathcal{T}} \tilde{p}(u), \tag{3}$$

where $\tilde{p}(u)$ denotes the probability that prefix u matches the sampled path. This reduces tree construction to selecting prefixes with large probability mass. Since $\tilde{p}(u)$ factorizes along the prefix,

$$\log \tilde{p}(u) = \sum_{i=1}^{|u|} \log \tilde{p}(u_i \mid u_{<i}), \tag{4}$$

providing an additive scoring rule that enables priority-based expansion.

2.2. Prefix-Aligned Surrogate Distributions

Definition 2.1 (Prefix-Aligned Surrogate)**.** *A surrogate distribution $\tilde{p}$ over $\mathcal{V}^k$ is* prefix-aligned *if there exists some position $i \ge 2$ such that the conditional distribution $\tilde{p}(u_i \mid u_{<i})$ is not invariant to the prefix, i.e., there exist $u_{<i} \ne u'_{<i}$ with*

$$\tilde{p}(u_i \mid u_{<i}) \ne \tilde{p}(u_i \mid u'_{<i}). \tag{5}$$

The nontriviality clause is essential. Under an independence assumption, any joint distribution admits a trivial prefix-blind factorization of the form $\tilde{p}(u) = \prod_i \tilde{p}_i(u_i)$, where each token is scored independently of the prefix. Such decompositions assign identical per-position contributions regardless of the realized prefix and therefore fail to capture the prefix-conditioned structure in Eq. (1).

2.3. Autoregressive vs. Diffusion Surrogate Distributions

Existing tree drafting methods for AR drafters implicitly assume autoregressive prefix-conditioned scores [13, 15, 19, 18]. This assumption fundamentally breaks under diffusion drafting.

Autoregressive drafting yields a prefix-aligned surrogate. An AR draft model p_D defines a prefix-conditioned distribution over sequences via the chain rule:

$$\tilde{p}_{\mathrm{AR}}(P) := p_D(P) = \prod_{i=1}^{k} p_D(x_i \mid x_{<i}). \tag{6}$$

Each conditional $p_D(x_i \mid x_{<i})$ is a genuine function of the prefix, a well-trained AR drafter approximates the target's prefix conditionals, so $\tilde{p}_{\mathrm{AR}}$ is prefix-aligned (Definition 2.1) and its log-probability factorization in Eq. (6) provides an effective surrogate score for tree construction.

Diffusion drafting yields a prefix-blind marginal surrogate. A diffusion drafter produces a collection of *position-wise marginals* $q_1, \ldots, q_k$ over $\mathcal{V}$. Each $q_i(x_i)$ is the marginal distribution of position i; it does not condition on the prefix $x_{<i} = (x_1, \ldots, x_{i-1})$.[1] The induced factorized distribution over candidate paths is

$$\tilde{p}_{\mathrm{diff}}(P) := \prod_{i=1}^{k} q_i(x_i). \tag{7}$$

The marginal surrogate $\tilde{p}_{\mathrm{diff}}$ is not prefix-aligned: each conditional $\tilde{p}_{\mathrm{diff}}(x_i \mid x_{<i}) = q_i(x_i)$ takes the same value regardless of $x_{<i}$, violating the nontriviality clause of Definition 2.1. The structural mismatch with prefix-based verification is *intrinsic* to the marginal factorization, not an artifact of any particular drafter or training procedure. In Section 3, we will show that while this mismatch does not preclude diffusion-based drafting from being effective, it can affect the quality of path ranking.

3. Observations

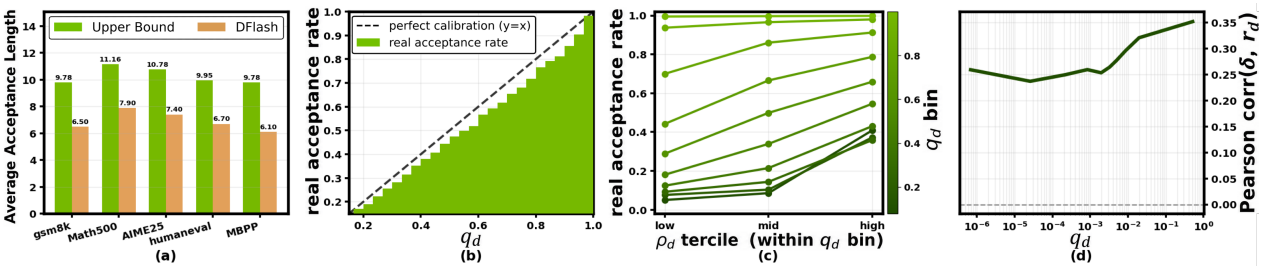


Figure 2 | (a) Linear drafting leaves room for improvement via multi-path exploration. (b) The predicted confidence from q_d closely matches the empirical acceptance rate. (c) Within fixed q_d bins, empirical acceptance rates increase from low to high ρ_d terciles, indicating that prefix-conditioned compatibility captures predictive signal beyond the marginal confidence q_d. (d) ρ_d provides a correction direction aligned with the error of q_d, although it remains noisy (correlation < 1).

In this section, we use Qwen3-4B-DFlash with mask block size being 16 as an example to analyze the characteristics of diffusion-based draft scoring. To study the impact of tree construction, we adopt the tree expansion strategy from EAGLE [15, 16] as the default tree construction algorithm. For each candidate token, we record its diffusion draft probability q_d, prefix-conditioned score ρ_d, and final acceptance outcome. The prefix-conditioned score ρ_d is instantiated using an n-gram model to capture compatibility with prefix-based AR verification. Unless otherwise specified, results are aggregated across datasets.

Observation 1: Linear drafting leaves substantial room for improvement through multi-path exploration. We first compare the acceptance length achieved by linear drafting with an oracle upper bound equivalent to exhaustive multi-path exploration. In our setting, the diffusion drafter generates a block of 16

[1] The dependence on the prompt and previously generated context, encoded in a shared latent state, is suppressed in this section for notational clarity.

draft positions simultaneously, naturally exposing a rich multi-candidate draft space within a single forward pass. To estimate the achievable upper bound, at each position, we retain the top-10 candidates from the diffusion marginal distribution and perform a position-wise oracle matching analysis. Specifically, we check whether the ground-truth token at each position appears in the corresponding top-10 candidate set, and then compute the longest contiguous prefix length covered by these matches.[2] As shown in Figure 2(a), a substantial gap consistently exists between linear drafting and this upper bound across all tasks. For example, on GSM8K the acceptance length improves from 6.5 to 9.78, while on Math500 it increases from 7.9 to 11.16. Similar trends are observed on AIME25, HumanEval, and MBPP. These results suggest that existing linear drafting strategies fail to fully exploit the rich combinatorial candidate space already produced by diffusion drafters, leaving significant room for improvement via multi-path exploration.

Observation 2: Marginal confidence correlates with overall acceptance. We further examine whether the diffusion drafter provides informative estimates of downstream verification success. Specifically, conditioned on the preceding prefix being accepted by the verifier, we compute the diffusion marginal confidence q_d for the next draft token using the exponentiated marginal probability under the diffusion distribution, and compare it against the empirical acceptance rate observed during verification. As shown in Figure 2(b), the acceptance rate increases monotonically with q_d, demonstrating a strong positive correlation between diffusion confidence and actual verification success. This suggests that the diffusion marginal distribution already captures meaningful global acceptance trends, and that higher-confidence draft tokens are substantially more likely to be accepted. At the same time, the calibration curve consistently remains below the ideal $y = x$ line, indicating that marginal confidence alone is still imperfect and does not fully characterize verifier acceptance. Nevertheless, q_d provides a strong global signal for estimating acceptance likelihood and serves as a useful foundation for candidate ranking.

Observation 3: Marginal signals are insufficient but can be improved by prefix conditioning. Despite its strong global correlation with acceptance, q_d alone is insufficient for reliable token-level ranking: as shown in Figure 2(c), tokens with similar q_d can yield substantially different acceptance outcomes when grouped by ρ_d. This implies that q_d is not a sufficient statistic for predicting acceptance, and that ρ_d provides complementary, prefix-conditioned information. This additional prefix-conditioned signal also remains directionally informative: Figure 2(d) shows that the correction term $\delta = \log \rho_d - \log q_d$ positively correlates with the true residual $r_q = \log p_T - \log q_d$. This indicates that ρ_d provides a correction direction that aligns with the true error of the draft model. Importantly, the correlation remains significantly below 1, indicating that ρ_d is not a direct proxy for the target distribution. Instead, it acts as a noisy but directionally aligned correction signal.

Insights. The draft probability q_d provides informative estimates of global acceptance, but is still insufficient for accurate path ranking. The prefix-conditioned signal ρ_d offers complementary information that aligns with the error of q_d, but is itself noisy. Together, these observations suggest that *an effective scoring should preserve q_d while incorporating a controlled correction based on ρ_d.*

4. PRESTO

We now introduce PRESTO, a tree-based diffusion drafting framework that augments diffusion drafter with prefix-aligned information, enabling more effective candidate exploration under prefix-conditioned AR verification. In Section 4.1, we introduce prefix-aligned scoring that addresses the aforementioned structural mismatch. In Section 4.2, we describe tree drafting via priority-based tree search. In Section 4.3, we elaborate how to extend tree drafting to self-speculative dLLMs.

4.1. Prefix-Aligned Scoring via Minimal Correction

From surrogate distribution to scoring function. Section 2 reduces tree construction to ranking prefixes by $\log \tilde{p}(u)$ (Eq. (4)). For diffusion drafting, the natural marginal surrogate $\tilde{p}_{\text{diff}}(P) = \prod_i q_i(x_i)$ is prefix-blind (Eq. (7)), creating a mismatch with prefix-based verification. We therefore seek a prefix-aligned surrogate $\tilde{p}^\star$

[2]The reported upper bound is equivalent to the result that would be obtained by exhaustive enumeration. However, we do not explicitly enumerate all 10^{16} paths. Instead, we exploit the fact that acceptance is determined by whether the ground-truth token is covered by the top-k candidate set at each position. This allows us to compute the same oracle upper bound efficiently through position-wise matching.

that preserves the calibrated drafter signal while incorporating a prefix-conditioned correction in terms of the observations and insights from Section 3.

Prefix-aligned surrogate. Let $q_d(t)$ denote the diffusion drafter's marginal at position d, and let $\rho_d(t \mid c_d)$ denote a tractable prefix-conditioned signal, where $c_d = (t_1, \ldots, t_{d-1})$ is the within-block prefix. We define the prefix-aligned conditional as a multiplicative combination:

$$p_d^\star(t \mid c_d) \propto q_d(t)\, \rho_d(t \mid c_d)^{\lambda_d}, \tag{8}$$

where $\lambda_d \geq 0$ controls the strength of the prefix correction. The induced joint surrogate distribution is

$$\tilde{p}^\star(P) := \prod_{d=1}^{k} p_d^\star(t_d \mid c_d), \tag{9}$$

which is prefix-aligned (Definition 2.1) whenever $\lambda_d > 0$ and ρ_d depends nontrivially on c_d. Eq. (9) thereby provides a candidate surrogate that addresses the structural mismatch of $\tilde{p}_{\text{diff}}$. This form can also be interpreted as the solution to a KL-regularized objective that balances closeness to q_d with alignment to ρ_d (see Appendix B).

For simplicity, we use the unnormalized log-score induced by Eq. (8). Specifically, we define the token-level score as $s_{d,t}(c_d) = \log q_d(t) + \lambda_d \log \rho_d(t \mid c_d)$, and the path score

$$S(P) = \sum_{d=1}^{k} s_{d,t_d}(c_d), \qquad c_d = (t_1, \ldots, t_{d-1}). \tag{10}$$

Eq. (10) decomposes additively over depth, supporting incremental priority-based tree expansion.

4.2. Priority-Based Tree Construction

Algorithm 1 Beam Search with Global Retention

```
procedure BeamSearch(s, B, b, W, D)
    P ← {root}, T ← {root}
    for d = 1 to D while |T| < B do
        P_new ← ∅
        for each v ∈ P do
            C ← Expand(v, b, s)
            P_new ← P_new ∪ C
            T ← T ∪ C
        end for
        P ← TopW(P_new, W)
    end for
    return TopB(T, B)
end procedure
```

Algorithm 2 Best-First Search

```
procedure BestFirst(s, B, b)
    P ← {root}, T ← {root}
    S(root) ← 0
    while |T| < B do
        v* ← arg max_{u∈P} S(u)
        P ← P \ {v*}
        C ← Expand(v*, b, s)
        Insert C into priority queue P
        T ← T ∪ C
    end while
                                  ▷ tree construction complete
    return TopB(T, B)
end procedure
```

From the surrogate objective to priority-based expansion. Section 2 shows that maximizing the surrogate objective in Eq. (3) reduces to selecting the top-B prefixes by their log-probabilities under $\tilde{p}^\star$. Equivalently, the optimal tree consists of the B prefixes u with the largest $\log \tilde{p}^\star(u) = S(u)$, where S is the path score in Eq. (10). However, exhaustively expanding and ranking all candidate continuations is computationally infeasible due to the exponentially growing search space and large vocabulary branching factor. Instead, since the path score decomposes additively over depth, high-scoring partial prefixes are more likely to remain among the top-ranked candidates after further expansion. This naturally motivates a greedy priority-based expansion strategy for diffusion drafting: maintain a frontier of expandable nodes and iteratively expand the highest-scoring node.

We identify each tree node v with the unique path from the root to v, denoted $P(v) = (t_1, \ldots, t_{d(v)})$, where $d(v) := |P(v)|$ is the depth of v. The cumulative path score is $S(v) := S(P(v)) = \sum_{i=1}^{d(v)} s_{i,t_i}(c_i)$ with $c_i = (t_1, \ldots, t_{i-1})$. The root has $d(\text{root}) = 0, S(\text{root}) = 0$. At each iteration, the algorithm:

1. Selects a frontier node $v^\star$ according to a search policy π;
2. Expands $v^\star$ by adding its top-b children, ranked by the token score $s_{d(v^\star)+1,t}(c_{d(v^\star)+1})$ at depth $d(v^\star)+1$;
3. Updates the frontier with the newly added children, removing $v^\star$ from the expandable set.

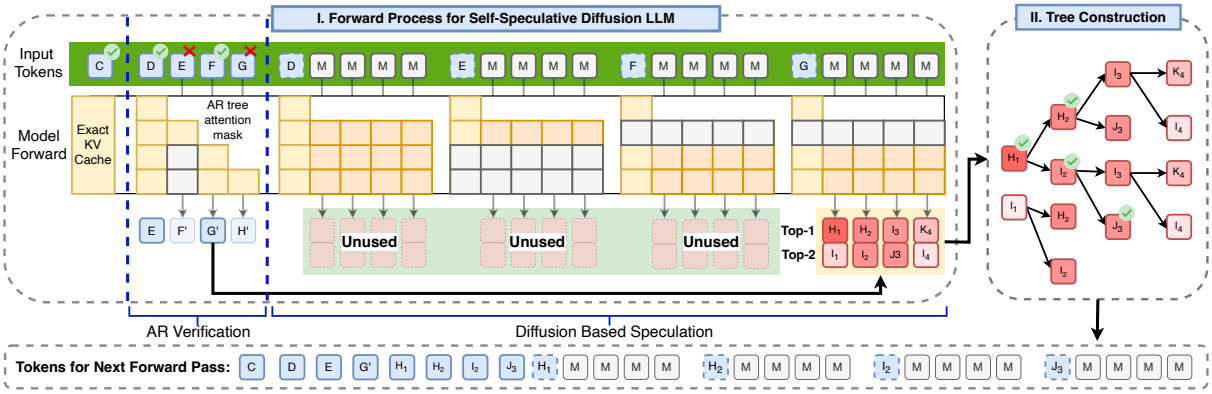


Figure 3 | Application of PRESTO to hybrid dLLMs under self-speculative decoding with a tree budget of 4 drafting tokens. The committed prefix reuses the exact KV cache for AR verification of the previous draft tree, while trailing mask-token slots are simultaneously denoised for diffusion-based speculation within the same forward pass. Speculative outputs attached to rejected branches are discarded, while valid speculation slots along the accepted path are used for next-round tree construction. The resulting draft tree is flattened into draft tokens for the next forward pass under AR tree attention. PRESTO is the first framework to explore tree-based drafting for self-speculative dLLMs. The detailed process is provided in Appendix C.

The iteration terminates when $|\mathcal{T}| = B$. Because S is additive over depth, each child's cumulative score is computed incrementally from its parent's, avoiding redundant computation.

We consider two widely used instantiations of priority-based expansion under the path score S: beam search with global retention (Algorithm 1) and best-first search (BFS) (Algorithm 2). Both prioritize high-S partial paths but differ in how the expansion budget is allocated across depth: BFS maintains a global priority over the entire frontier, while beam search restricts expansion to the top-W in-beam nodes per depth and retains out-of-beam nodes for the final top-B selection. Beam search can thus be interpreted as a width-constrained approximation to BFS.

Table 1 | Acceptance length comparison.

	Qwen3-4B-DFlash		Qwen3-8B-DFlash	
Budget	BFS	Beam	BFS	Beam
$B = 128$	9.21	9.27	9.25	9.28
$B = 256$	9.55	9.54	9.60	9.62
$B = 512$	9.81	9.79	9.90	9.88
$B = 1024$	9.98	9.85	10.10	9.93

BFS vs. Beam Search. Table 1 reports acceptance length under the two policies across budget sizes. At small to moderate budgets ($B \leq 512$), best-first and beam search yield similar acceptance length, as beam search's per-depth budget is sufficient to cover most high-priority expansions that best-first search would select under global prioritization. At $B = 1024$, best-first outperforms beam search by a small margin: with a larger budget, the preferred expansion pattern can become less uniform across depth, which a global priority handles more flexibly than a fixed per-depth beam. In practice, verification shifts from memory-bound to compute-bound once B exceeds a hardware-dependent threshold, after which forward-pass cost grows linearly with B and erodes the speedup. We therefore operate at $B \leq 512$, which sits within the memory-bound regime where the two policies are comparable, and adopt beam search as the default search strategy.

4.3. Extending Tree Construction to hybrid dLLMs

The proposed tree-construction framework can be naturally extended to support quadratic self-speculative decoding in hybrid dLLMs [8, 12][3], as illustrated in Figure 3. Quadratic self-speculation makes autoregressive verification and diffusion-based drafting parallelized within a single model forward pass: previously drafted tokens are verified autoregressively, while new draft candidates are simultaneously generated at trailing mask

[3]Nemotron-Labs-Diffusion primarily highlights a linear self-speculation decoding mode, where the model first generates draft tokens through diffusion decoding and then verifies them autoregressively using the same model. Applying PRESTO to this setting is straightforward and identical to the dedicated diffusion drafter setting discussed earlier, as PRESTO only modifies the diffusion drafting process while leaving the verification procedure unchanged. In addition to linear self-speculation, the Nemotron-Labs-Diffusion paper also introduces a quadratic self-speculation variant [12] which is further explored in this work.

positions through diffusion decoding.

PRESTO preserves this original verify-and-draft pipeline without introducing additional model forwards. During each forward pass, the committed prefix reuses the exact KV cache for AR verification of the previous draft tree, while the trailing mask-token slots attached to speculative branches are simultaneously denoised for next-round drafting. After verification, only the mask slots attached to the longest accepted path remain valid speculation positions, while the slots attached to rejected branches are discarded since their predictions were conditioned on prefixes that are no longer part of the committed context. PRESTO then extracts top-k diffusion candidates from the valid mask slots and organizes them into a new draft tree using the proposed prefix-aligned scoring and priority-based tree expansion. The resulting tree is flattened into draft tokens and verified in the next forward pass using the AR tree attention mechanism.

This extension is particularly well-suited for self-speculative dLLMs because in standard linear self-speculative decoding, speculative tokens attached to rejected branches are discarded after verification, leading to substantial wasted computation. PRESTO alleviates this inefficiency by organizing diffusion outputs into a tree structure, allowing diverse speculative candidates generated within the same forward pass to be more effectively utilized during verification. As shown in Section 5.2, replacing linear drafting with PRESTO consistently improves acceptance length and decoding throughput across different datasets and model sizes.

5. Empirical Validation

In this section, we demonstrate that PRESTO significantly improves the efficiency of diffusion-based speculative decoding across both standard and self-speculative diffusion based SD.

5.1. Experiment Setup

Models and Tasks. We evaluate PRESTO on two settings: (i) standard speculative decoding with a separate diffusion drafter and AR target: dFlash with Qwen3-4B, Qwen3-8B, and Qwen3-Coder-30B-A3B backbones; and (ii) hybrid dLLM: Nemotron-Labs-Diffusion-8B at both linear self-speculation and quadratic self-speculation modes. Evaluation spans three task categories: **mathematical reasoning** (GSM8K [20], MATH [21], AIME24 [22], AIME25 [23]), **code** (HumanEval [24], MBPP [25], LiveCodeBench [26]), and **conversation** (MT-Bench [27], Alpaca [28]). We report average acceptance length (τ) and throughput (token/s) speedup over the AR baseline. All experiments run on NVIDIA B200 GPUs.

Implementations. We implement PRESTO using two runtime backends. For dFlash and Nemotron-Labs-Diffusion under quadratic self-speculation, we use a PyTorch implementation with FlexAttention [29] for target model verification with tree-structured attention mask [13, 14, 15]. For Nemotron-Labs-Diffusion under linear self-speculation, we integrate PRESTO into the official SGLang implementation with FlashInfer attention [30]. We instantiate the prefix-aligned score using a lightweight n-gram model with $n = 3$[4]. Unless otherwise specified, we set $\lambda_d = 0.2$, batch size to 1, and use beam search with global retention, using beam width 10 and top-k expansion ($k = 10$) per position. We evaluate PRESTO under both deterministic ($T = 0$) and stochastic ($T = 1$) decoding. For dFlash, we sweep tree budgets $\mathcal{T} \in \{128, 256, 512\}$ and report the best-performing configuration in terms of end-to-end throughput. For Nemotron-Labs-Diffusion, we use a fixed tree budget $\mathcal{T}$ of 32 following its speculative block size.

5.2. Experiment Results

PRESTO on dFlash. As illustrated in Table 2, PRESTO consistently achieves substantial improvements in the average acceptance length τ across all models and tasks, which directly translates into higher end-to-end throughput speedup. For example, on Qwen3-4B with $T = 0$, τ increases from 7.9 to 10.9 on MATH-500 and from 7.4 to 10.3 on AIME25, while on code benchmarks it improves from 6.9 to 10.0 on LCB and from 6.7 to 9.9 on HumanEval, leading to corresponding speedup gains across all settings. The largest improvements are observed on math and code benchmarks, where τ typically increases by $+2$ to $+3$ tokens. These trends remain stable across model scales and decoding regimes: even for stronger drafters such as Qwen3-8B and under stochastic decoding ($T = 1$), PRESTO continues to deliver comparable relative gains. These results

[4]Following the n-gram model construction in [31].

Table 2 | Decoding speedup over baseline and average acceptance length (τ) on Qwen3 models with thinking mode disabled and a maximum of 2048 generated tokens.

Model	Method	Math								Code								Chat				*Avg.*	
		GSM8K		MATH-500		AIME24		AIME25		HumanEval		MBPP		LCB		SWE-Bench		MT-Bench		Alpaca		*Avg.*	
Temperature = 0		Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ
4B	dFlash	5.1×	6.5	6.5×	7.9	6.5×	7.4	6.2×	7.4	5.6×	6.7	5.0×	6.1	5.9×	6.9	3.2×	3.6	2.9×	4.4	2.4×	3.1	4.9×	6.0
	PRESTO	7.9×	9.4	9.2×	10.9	8.7×	10.5	8.8×	10.3	8.1×	9.9	7.9×	9.3	8.4×	10.0	5.3×	5.9	4.9×	6.7	3.9×	5.0	7.3×	8.8
	× **Gain**	**1.55×**		**1.42×**		**1.34×**		**1.42×**		**1.45×**		**1.58×**		**1.42×**		**1.66×**		**1.69×**		**1.62×**		**1.48×**	
8B	dFlash	5.0×	6.6	6.3×	7.9	6.2×	7.5	6.0×	7.0	5.4×	6.5	4.8×	6.0	5.7×	7.1	3.3×	3.6	2.8×	4.3	2.4×	3.1	4.8×	6.0
	PRESTO	8.0×	9.6	9.4×	11.1	8.5×	10.7	8.7×	10.2	8.5×	9.9	7.7×	9.4	8.5×	10.3	5.0×	6.0	4.8×	6.6	3.9×	5.0	7.3×	8.9
	× **Gain**	**1.60×**		**1.49×**		**1.37×**		**1.45×**		**1.57×**		**1.60×**		**1.49×**		**1.52×**		**1.71×**		**1.62×**		**1.52×**	
30B	dFlash	3.1×	5.2	4.1×	5.6	2.8×	5.3	2.7×	5.1	5.6×	8.0	5.6×	7.2	3.6×	6.2	3.2×	3.6	2.2×	3.5	1.8×	2.2	3.5×	5.2
	PRESTO	4.1×	7.9	5.4×	8.0	3.6×	8.0	3.6×	7.9	7.0×	10.9	7.5×	9.9	4.7×	8.5	4.6×	5.6	3.0×	5.2	2.5×	3.4	4.6×	7.5
	× **Gain**	**1.32×**		**1.32×**		**1.29×**		**1.33×**		**1.25×**		**1.34×**		**1.31×**		**1.44×**		**1.36×**		**1.39×**		**1.33×**	
Temperature = 1		Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ
4B	dFlash	4.7×	6.0	5.2×	6.6	3.8×	5.0	3.9×	4.9	4.8×	6.0	4.4×	5.6	5.0×	6.6	2.5×	3.1	2.7×	4.1	2.2×	3.0	3.9×	5.1
	PRESTO	7.2×	9.1	7.6×	9.8	6.0×	7.7	6.1×	7.7	7.6×	9.3	7.0×	9.0	6.7×	9.4	4.2×	5.0	4.3×	6.2	3.7×	4.7	6.0×	7.8
	× **Gain**	**1.53×**		**1.46×**		**1.58×**		**1.56×**		**1.58×**		**1.59×**		**1.34×**		**1.68×**		**1.59×**		**1.68×**		**1.54×**	
8B	dFlash	4.8×	6.0	5.0×	6.6	3.9×	5.1	3.8×	5.0	4.4×	5.4	4.1×	5.2	5.2×	6.8	2.3×	2.8	2.6×	3.8	2.1×	2.9	3.8×	5.0
	PRESTO	7.4×	9.0	7.4×	9.6	6.3×	7.8	6.5×	7.7	7.2×	8.6	6.9×	8.6	7.0×	9.8	4.0×	4.7	4.2×	5.9	3.6×	4.7	6.0×	7.6
	× **Gain**	**1.54×**		**1.48×**		**1.62×**		**1.71×**		**1.64×**		**1.68×**		**1.35×**		**1.74×**		**1.62×**		**1.71×**		**1.58×**	
30B	dFlash	4.1×	5.1	4.2×	5.3	3.3×	4.3	3.3×	4.2	5.8×	7.7	5.6×	7.1	3.8×	5.6	2.7×	3.2	2.1×	3.2	1.8×	2.1	3.7×	4.8
	PRESTO	4.5×	7.8	5.5×	7.7	3.7×	6.7	3.4×	6.5	6.8×	10.2	7.9×	9.6	4.3×	8.0	4.4×	4.9	3.0×	5.0	2.6×	3.3	4.6×	7.0
	× **Gain**	**1.10×**		**1.31×**		**1.12×**		**1.03×**		**1.17×**		**1.41×**		**1.13×**		**1.63×**		**1.43×**		**1.44×**		**1.26×**	

indicate that PRESTO effectively pushes the acceptance length of dFlash closer to its upper bound, delivering consistent and substantial end-to-end throughput speedup gains. The end-to-end throughput results based on torch implementation is shown in Appendix D.

Table 3 | Decoding speedup over baseline, average acceptance length (τ), and absolute throughput on Nemotron-Labs-Diffusion-8B at linear self speculation decoding mode with a maximum of 2048 generated tokens.

Model	Method	Math								Code								Chat				*Avg.*		
		GSM8K		MATH-500		AIME24		AIME25		HumanEval		MBPP		LCB		SWE-Bench		MT-Bench		Alpaca		*Avg.*		
Temperature = 0		Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Thr.
8B	vanilla	5.7×	10.8	6.4×	12.2	5.7×	11.0	5.5×	10.5	5.0×	9.3	4.4×	8.2	4.7×	9.0	3.7×	6.8	2.8×	5.2	2.5×	4.6	4.6×	8.8	642
	PRESTO	6.0×	11.9	6.7×	13.2	6.1×	12.2	5.8×	11.5	5.2×	10.3	4.6×	9.1	4.9×	10.0	4.0×	7.9	3.2×	6.5	2.9×	5.8	4.9×	9.9	686
	× **Gain**	**1.05×**		**1.04×**		**1.07×**		**1.06×**		**1.04×**		**1.04×**		**1.04×**		**1.08×**		**1.14×**		**1.16×**		**1.06×**		
Temperature = 1		Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Thr.
8B	vanilla	2.9×	5.6	2.9×	5.7	2.5×	5.0	2.6×	4.9	2.3×	4.6	2.1×	4.2	2.0×	4.0	1.4×	2.9	1.4×	2.7	1.2×	2.3	2.1×	4.2	290
	PRESTO	3.1×	6.1	3.3×	7.1	2.9×	5.7	2.9×	6.4	2.7×	5.5	2.6×	5.0	2.3×	4.8	1.8×	3.6	1.8×	3.5	1.5×	3.2	2.5×	5.1	339
	× **Gain**	**1.07×**		**1.14×**		**1.16×**		**1.12×**		**1.17×**		**1.24×**		**1.15×**		**1.29×**		**1.29×**		**1.25×**		**1.17×**		

PRESTO on Nemotron-Labs-Diffusion (NLD). As shown in Table 3, PRESTO consistently improves both throughput and average acceptance length over the official linear self-speculative decoding implementation of Nemotron-Labs-Diffusion across all benchmarks and decoding temperatures. By replacing linear drafting with tree-based exploration while preserving the original verification pipeline, PRESTO remains effective across a wide range of tasks. Under deterministic decoding ($T = 0$), PRESTO increases average end-to-end throughput from 4.6× to 4.9× while improving average acceptance length from 8.8 to 9.9, with consistent gains observed on all benchmarks. Under stochastic decoding ($T = 1$), where a single drafted trajectory is more likely to diverge from the target model, the benefits become more pronounced: average end-to-end throughput improves from 2.1× to 2.5×, while average acceptance length increases from 4.2 to 5.1. Across individual benchmarks, PRESTO achieves up to 1.29× end-to-end throughput improvement and increases acceptance length by as much as 39% on Alpaca under stochastic decoding (T=1). These results demonstrate that the benefits of tree-based drafting persist even within a production-grade, highly optimized inference engine. The end-to-end throughput results based on SGLang is shown in Appendix E.

Table 4 | Decoding speedup over baseline, average acceptance length (τ), and absolute throughput on Nemotron-Labs-Diffusion-8B at quadratic self speculation decoding mode with a maximum of 2048 generated tokens.

Model	Method	Math								Code								Chat				*Avg.*		
		GSM8K		MATH-500		AIME24		AIME25		HumanEval		MBPP		LCB		SWE-Bench		MT-Bench		Alpaca		*Avg.*		
Temperature = 0		Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Thr.
8B	vanilla	2.6×	4.7	2.7×	6.8	3.2×	5.3	3.3×	5.7	3.2×	4.8	3.1×	4.5	2.4×	4.7	1.8×	3.5	2.1×	3.0	1.8×	2.7	2.6×	4.7	94
8B	PRESTO	5.3×	6.5	4.4×	7.5	3.5×	6.1	3.7×	6.4	3.7×	5.8	3.7×	5.4	3.1×	5.6	2.7×	4.7	2.8×	4.2	2.3×	3.6	3.5×	5.6	127
	× **Gain**	**2.04×**		**1.63×**		**1.09×**		**1.12×**		**1.16×**		**1.19×**		**1.29×**		**1.50×**		**1.33×**		**1.28×**		**1.35×**		
Temperature = 1		Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Sp.	τ	Thr.
8B	vanilla	1.4×	3.0	1.8×	3.4	1.5×	2.6	1.7×	3.0	1.7×	2.6	1.6×	2.4	1.2×	2.3	1.0×	1.8	1.2×	1.7	1.0×	1.5	1.4×	2.4	51
8B	PRESTO	4.1×	5.7	4.0×	6.6	2.9×	5.1	3.3×	5.7	3.2×	4.9	3.0×	4.4	2.8×	4.8	2.2×	3.7	2.3×	3.5	2.0×	3.0	3.0×	4.7	109
	× **Gain**	**2.93×**		**2.22×**		**1.93×**		**1.94×**		**1.88×**		**1.88×**		**2.33×**		**2.20×**		**1.92×**		**2.00×**		**2.14×**		

PRESTO on Nemotron-Labs-Diffusion (NLD) at quadratic self speculation mode. We further evaluate PRESTO under the quadratic self-speculative decoding setting introduced in Nemotron-Labs-Diffusion. As Table 4 shows, across all benchmarks and decoding settings, PRESTO also consistently outperforms vanilla quadratic self-speculative decoding in both end-to-end throughput and average acceptance length. Under deterministic decoding ($T = 0$), PRESTO provides stable gains across math, code, and chat tasks, improving average speedup from 2.6× to 3.5× on NLD-8B, while also consistently increasing acceptance length from 4.7 to 5.6. The largest gains are observed on math benchmarks, where PRESTO achieves up to 2.04× on GSM8k for NLD-8B. Gains are largest under stochastic decoding ($T = 1$), where single-path drafting degrades sharply from trajectory mismatch and accumulated uncertainty, while PRESTO 's tree-structured exploration retains multiple plausible continuations. On NLD-8B, this raises average end-to-end speedup from 1.4× to 3.0× and nearly doubles acceptance length from 2.4 to 4.7, reaching up to 2.9× throughput and over 2× acceptance length over vanilla self-speculative decoding. These positive results indicate the significant benefits of applying PRESTO to quadratic self-speculation.

We note that the throughput for dFlash and NLD quadratic self speculation can be further improved through system-level optimizations, including custom kernels, optimized KV-cache management, and efficient request scheduling. The goal here is not to fully optimize absolute throughput, but rather to provide a first-hand comparison between linear and tree-based drafting for diffusion drafters under the same native PyTorch implementation.

6. Discussions and Ablations

Effectiveness of prefix-conditioned signal. We first ablate the contribution of the prefix-conditioned compatibility term ρ_d by comparing our full scoring function against a variant that only relies on the diffusion marginal confidence q_d. As shown in Figure 4 top, incorporating ρ_d consistently improves the average acceptance length across all tasks and tree token budgets. The gains are particularly noticeable on reasoning and coding benchmarks such as GSM8K, Math500, and HumanEval, where prefix consistency plays a more important role in determining downstream verification success. These results suggest that diffusion marginal confidence alone is insufficient for reliably ranking candidate paths, while the prefix-conditioned signal provides complementary information that better aligns tree expansion with the autoregressive verification process.

Sensitivity of λ_d. Figure 4 middle shows the impact of varying λ_d under different datasets and tree token budgets. Across nearly all settings, moderate values of λ_d consistently achieve the best acceptance length, with $\lambda_d \approx 0.2$ providing the most stable performance. When λ_d is too small, the prefix-conditioned signal is underutilized and the scoring function degenerates toward pure diffusion confidence ranking. In contrast, overly large λ_d overemphasizes ρ_d and suppresses the diffusion marginal confidence q_d, leading to unstable candidate ranking and substantial degradation in acceptance length. Importantly, the optimal range remains highly consistent across different tasks and tree sizes, suggesting that the proposed scoring function is robust.

Entropy Adaptive λ_d vs. Fixed λ_d. We further explore an entropy-adaptive λ_d that dynamically balances q_d and ρ_d according to the uncertainty of the diffusion drafter: confident predictions rely more on q_d, while uncertain predictions assign larger weights to ρ_d (details in Appendix F). As shown in Figure 4 top, entropy-

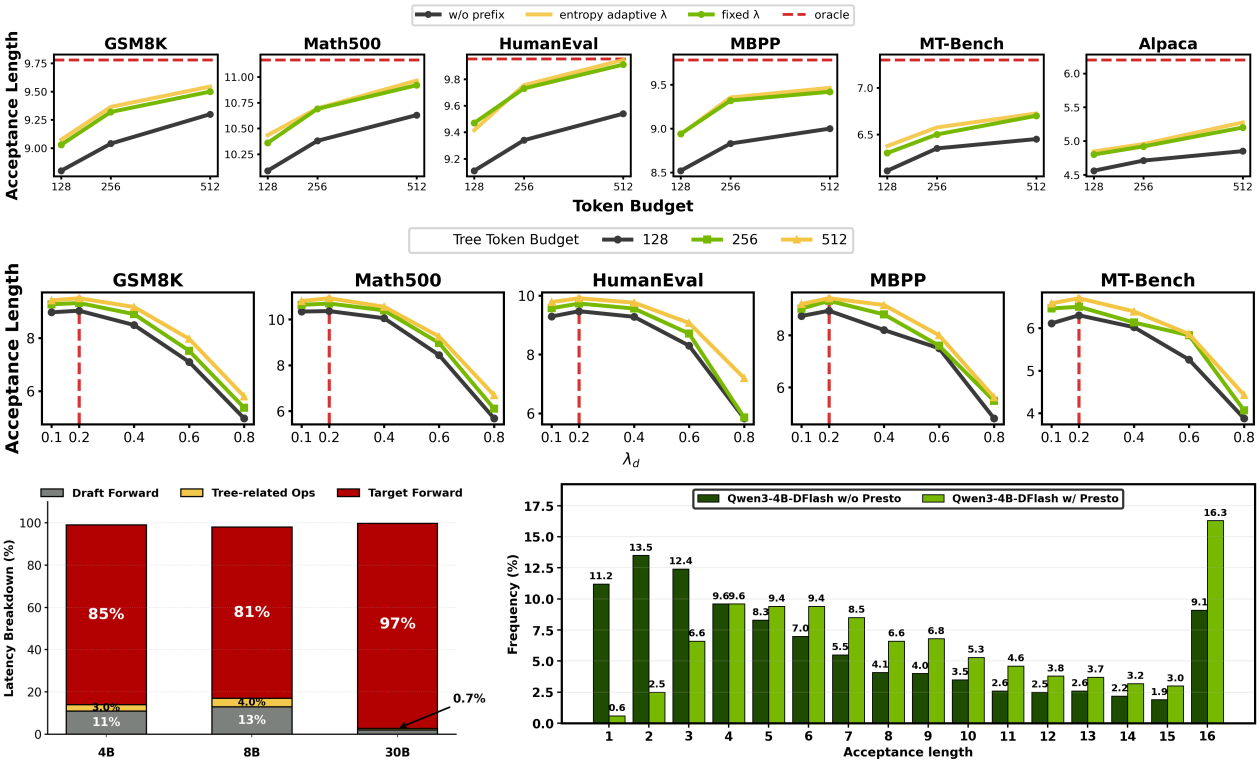


Figure 4 | **Top**: Effectiveness of prefix-conditioned compatibility signals. Incorporating the prefix-conditioned term ρ_d consistently improves acceptance length across datasets and tree token budgets. **Middle**: Sensitivity analysis of λ_d under different datasets and tree token budgets. Moderate values of λ_d consistently achieve the best performance, with $\lambda_d \approx 0.2$ providing the most stable gains across settings. **Bottom left**: Per-step latency breakdown across different model scales. Target-model verification dominates the overall decoding cost, while tree-related operations introduce only negligible overhead. **Bottom right**: Distribution of acceptance lengths on GSM8K. PRESTO significantly shifts the distribution toward longer accepted prefixes and substantially increases the frequency of near-complete and full-block acceptance. All results are obtained using Qwen3-4B-DFlash with diffusion block size 16.

adaptive λ_d provides only marginal improvements over a fixed λ_d, and the overall trends remain highly similar across datasets. This suggests that although uncertainty-aware weighting is intuitively appealing, a simple fixed λ_d is already sufficient to capture most of the benefits of prefix-conditioned ranking in practice, while avoiding additional complexity.

Overhead Analysis. As shown in Figure 4 bottom left, target-model verification overwhelmingly dominates the overall decoding latency across all model scales, accounting for 81%–97% of the total runtime. In comparison, tree-related operations, including tree construction, candidate filtering, and attention-mask preparation, contribute only 0.7%–4% of the per-step latency. This demonstrates that the proposed tree construction introduces negligible system overhead relative to the target-model forward pass. Moreover, the overhead becomes proportionally smaller for larger models, indicating favorable scalability of the framework.

Acceptance Length Distribution Analysis. Figure 4 bottom right further illustrates how PRESTO reshapes the acceptance-length distribution. Compared with vanilla diffusion drafting, PRESTO substantially reduces short-acceptance cases while significantly increasing the frequency of long-prefix and full-block acceptance. In particular, the probability of achieving near-complete or full-block acceptance increases dramatically, indicating that the proposed prefix-conditioned tree construction is more effective at identifying globally consistent candidate paths rather than locally confident but incompatible drafts. This distributional shift directly explains the throughput improvements observed in the main results.

7. Related Work

7.1. Speculative Decoding

Speculative decoding [32, 33] accelerates AR models by drafting tokens with a fast proposer and verifying them in parallel with the target model, provably preserving the target distribution. This guarantee relies on the target model possessing a well-trained verify distribution. A line of follow-up work eliminates the external drafter or improves draft quality: Medusa [14] augments the base LLM with multiple prediction heads and uses tree attention for parallel verification; the EAGLE family [34, 15, 16] exploits feature-level context from the frozen target model, with EAGLE-1 predicting future hidden states to boost acceptance, EAGLE-2 introducing adaptive drafting trees, and EAGLE-3 refining training objectives to scale speedups; and SpecInfer [13] proposes tree-structured verification. Multi-token prediction (MTP) [35] trains models to predict multiple future tokens simultaneously. Despite these advances, most existing methods rely on autoregressive drafting, which remains inherently sequential and limits attainable speedups.

7.2. Tree Construction in Speculative Decoding

A complementary line of work focuses on how draft tokens are organized for parallel verification. Medusa [14] popularized tree-structured drafting by packing multiple candidate continuations into a single token tree and verifying them in one forward pass via a topology-aware attention mask. However, this tree mask is hand-crafted and static. SpecInfer [13] introduced token-tree verification with provable distribution preservation. Subsequent work moves beyond fixed shapes: Sequoia [18] formulates tree construction as a dynamic-programming problem, jointly optimizing the tree topology and a hardware-aware tree size for a given accelerator. SpecExec [17] pushes draft trees to massively parallel sizes (thousands of nodes) to amortize the cost of offloaded weights on consumer hardware based on Best First Search like tree construction algorihtm. EAGLE-2 [15] observes that draft acceptance is context-dependent and uses the calibrated confidence scores of the EAGLE drafter to construct a dynamic draft tree per step via beam search like tree expension. OPT-Tree [36] formalizes the objective as maximizing the expected acceptance length and searches for the adaptive tree structure that attains it under a node budget. These methods consistently show that careful tree drafting contributes a substantial fraction of end-to-end speedup. However, they all assume an autoregressive drafter that produces tokens sequentially with conditional probabilities along each branch, and extending tree construction to diffusion-based drafters, whose draft tokens are produced in parallel remains an open problem that our work directly addresses.

7.3. Diffusion-based Speculative Decoding

Recent work explores using diffusion models as drafters within speculative decoding, combining the parallelism of diffusion drafting with the quality guarantee of AR verification. TiDAR [8] jointly trains diffusion and autoregressive objectives in a sequence-level hybrid, enabling parallel "thinking" via diffusion and sequential "talking" via autoregressive decoding, though final generation quality is not yet lossless. DiffuSpec [9] and SpecDiff-2 [37] employ large pretrained dLLMs as speculative drafters, with inference-time search or train–test alignment to improve acceptance. However, these approaches primarily focus on improving the quality or alignment of the diffusion drafter itself, often relying on massive drafters (*e.g.*, 7B parameters), which incur substantial memory and latency overhead. While they achieve long acceptance lengths, the high drafting cost can offset the practical speedups in real-world serving scenarios. In contrast, PRESTO studies how to more effectively exploit the rich multi-position candidate space produced by diffusion drafting through prefix-aware tree construction and priority-based expansion, making it complementary to stronger diffusion drafters and existing diffusion-based speculative decoding systems. DFlash [10] proposes to employ a lightweight block-diffusion drafter conditioned on context features extracted from the target model, generating an entire block of draft tokens in a single forward pass and reporting over 6× lossless acceleration. Similarly, DART [31] performs parallel logit prediction over multiple masked positions and assembles drafts via N-gram-guided tree pruning. However, its tree construction relies on a fixed, hand-tuned scoring rule with hard-coded hyperparameters, which leaves no principled way to adapt the tree shape to different budgets, drafters, or target models without re-tuning. In contrast, PRESTO formulates tree construction as priority-based expansion under a unified path score, yielding a single algorithmic framework that naturally generalizes across different tree budgets and diffusion drafter designs.

8. Conclusion

In this work, we propose PRESTO, a principled framework for tree-based speculative decoding with diffusion language models. We identify a fundamental mismatch between diffusion draft scoring and prefix-based autoregressive verification: while diffusion probabilities provide strong marginal plausibility signals, they are inherently prefix-blind and therefore insufficient for reliable path ranking in tree-based drafting. To address this issue, we introduce a prefix-aligned scoring mechanism together with a priority-based tree search strategy, enabling effective exploration of high-quality candidate paths during diffusion drafting.

9. Limitations

PRESTO requires a tractable prefix-aligned signal ρ_d to complement the diffusion drafter's prefix-blind marginals. In this work, we employ a lightweight n-gram model, which is empirically effective, but only captures short-range lexical compatibility. Exploring richer prefix-conditioned signals while preserving low overhead remains an important future direction. We hope this motivates further efforts to inject prefix-aligned signals into diffusion marginals.

We additionally explored an entropy-adaptive variant of λ_d, but observed only marginal improvements over a fixed coefficient, suggesting that more sophisticated adaptation strategies may require learned or target-aware scheduling mechanisms.

Finally, we primarily evaluate PRESTO under single-request decoding settings using the native PyTorch implementation except NLD at linear self speculation decoding mode. Extending PRESTO to production-scale serving frameworks such as vLLM and SGLang, together with characterizing its behavior under larger batch sizes, longer contexts, and highly optimized serving systems, remains meaningful future work.

References

[1] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models, 2025.

[2] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7b: Diffusion large language models, 2025.

[3] Chengyue Wu, Hao Zhang, Shuchen Xue, Shizhe Diao, Yonggan Fu, Zhijian Liu, Pavlo Molchanov, Ping Luo, Song Han, and Enze Xie. Fast-dllm v2: Efficient block-diffusion llm, 2025.

[4] Tiwei Bie, Maosong Cao, Xiang Cao, Bingsen Chen, Fuyuan Chen, Kun Chen, Lun Du, Daozhuo Feng, Haibo Feng, Mingliang Gong, Zhuocheng Gong, Yanmei Gu, Jian Guan, Kaiyuan Guan, Hongliang He, Zenan Huang, Juyong Jiang, Zhonghui Jiang, Zhenzhong Lan, Chengxi Li, Jianguo Li, Zehuan Li, Huabin Liu, Lin Liu, Guoshan Lu, Yuan Lu, Yuxin Ma, Xingyu Mou, Zhenxuan Pan, Kaida Qiu, Yuji Ren, Jianfeng Tan, Yiding Tian, Zian Wang, Lanning Wei, Tao Wu, Yipeng Xing, Wentao Ye, Liangyu Zha, Tianze Zhang, Xiaolu Zhang, Junbo Zhao, Da Zheng, Hao Zhong, Wanli Zhong, Jun Zhou, Junlin Zhou, Liwang Zhu, Muzhi Zhu, and Yihong Zhuang. Llada2.1: Speeding up text diffusion via token editing, 2026.

[5] Shuang Cheng, Yihan Bian, Dawei Liu, Linfeng Zhang, Qian Yao, Zhongbo Tian, Wenhai Wang, Qipeng Guo, Kai Chen, Biqing Qi, and Bowen Zhou. Sdar: A synergistic diffusion-autoregression paradigm for scalable sequence generation, 2025.

[6] Yonggan Fu, Lexington Whalen, Zhifan Ye, Xin Dong, Shizhe Diao, Jingyu Liu, Chengyue Wu, Hao Zhang, Enze Xie, Song Han, Maksim Khadkevich, Jan Kautz, Yingyan Celine Lin, and Pavlo Molchanov. Efficient-dlm: From autoregressive to diffusion language models, and beyond in speed, 2026.

[7] Chengyue Wu, Hao Zhang, Shuchen Xue, Zhijian Liu, Shizhe Diao, Ligeng Zhu, Ping Luo, Song Han, and Enze Xie. Fast-dllm: Training-free acceleration of diffusion llm by enabling kv cache and parallel decoding, 2025.

[8] Jingyu Liu, Xin Dong, Zhifan Ye, Rishabh Mehta, Yonggan Fu, Vartika Singh, Jan Kautz, Ce Zhang, and Pavlo Molchanov. Tidar: Think in diffusion, talk in autoregression, 2025.

[9] Guanghao Li, Zhihui Fu, Min Fang, Qibin Zhao, Ming Tang, Chun Yuan, and Jun Wang. Diffuspec: Unlocking diffusion language models for speculative decoding, 2025.

[10] Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding, 2026.

[11] Yifan Yu, Yuqing Jian, Junxiong Wang, Zhongzhu Zhou, Donglin Zhuang, Xinyu Fang, Sri Yanamandra, Xiaoxia Wu, Qingyang Wu, Shuaiwen Leon Song, Tri Dao, Ben Athiwaratkun, James Zou, Fan Lai, and Chenfeng Xu. Introspective diffusion language models, 2026.

[12] Yonggan Fu, Lexington Whalen, Abhinav Garg, Chengyue Wu, Maksim Khadkevich, Nicolai Oswald, Enze Xie, Daniel Egert, Sharath Turuvekere Sreenivas, Shizhe Diao, Chenhan Yu, Ye Yu, Weijia Chen, Sajad Norouzi, Jingyu Liu, Shiyi Lan, Ligeng Zhu, Jin Wang, Jindong Jiang, Morteza Mardani, Mehran Maghoumi, Song Han, Ante Jukic, Nima Tajbakhsh, Jan Kautz, and Pavlo Molchanov. Nemotron-labs-diffusion: A tri-mode language model unifying autoregressive, diffusion, and self-speculation decoding. May 2026. Technical report.

[13] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, page 932–949. ACM, April 2024.

[14] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024.

[15] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees, 2024.

[16] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-3: Scaling up inference acceleration of large language models via training-time test, 2025.

[17] Ruslan Svirschevski, Avner May, Zhuoming Chen, Beidi Chen, Zhihao Jia, and Max Ryabinin. Specexec: Massively parallel speculative decoding for interactive llm inference on consumer devices, 2024.

[18] Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable, robust, and hardware-aware speculative decoding, 2025.

[19] Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, Ahmad Beirami, Himanshu Jain, and Felix Yu. Spectr: Fast speculative decoding via optimal transport, 2024.

[20] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.

[21] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023.

[22] Art of Problem Solving. Aime problems and solutions. https://artofproblemsolving.com/wiki/index.php/AIME, 2024. Accessed: 2025-04-20.

[23] MAA. American invitational mathematics examination (aime). https://maa.org/math-competitions/aime. Mathematics Competition Series; n.d.a.

[24] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.

[25] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021.

[26] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024.

[27] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.

[28] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. https://github.com/tatsu-lab/stanford_alpaca, 2023.

[29] Juechu Dong, Boyuan Feng, Driss Guessous, Yanbo Liang, and Horace He. Flex attention: A programming model for generating optimized attention kernels, 2024.

[30] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. Flashinfer: Efficient and customizable attention engine for llm inference serving, 2025.

[31] Fuliang Liu, Xue Li, Ketai Zhao, Yinxi Gao, Ziyan Zhou, Zhonghui Zhang, Zhibin Wang, Wanchun Dou, Sheng Zhong, and Chen Tian. Dart: Diffusion-inspired speculative decoding for fast llm inference, 2026.

[32] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding, 2023.

[33] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023.

[34] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty, 2025.

[35] Mohammad Samragh, Arnav Kundu, David Harrison, Kumari Nishu, Devang Naik, Minsik Cho, and Mehrdad Farajtabar. Your llm knows the future: Uncovering its multi-token prediction potential, 2025.

[36] Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. Opt-tree: Speculative decoding with adaptive draft tree structure, 2025.

[37] Jameson Sandler, Jacob K. Christopher, Thomas Hartvigsen, and Ferdinando Fioretto. Specdiff-2: Scaling diffusion drafter alignment for faster speculative decoding, 2025.

A. Prefix-Mass Decomposition of the Tree Objective

Let $\mathcal{V}$ denote the token vocabulary and let $\mathcal{T}$ be a prefix-closed tree whose nodes are finite token sequences. Define

$$\mathcal{T}^+ := \mathcal{T} \setminus \{\emptyset\}$$

as the set of non-root nodes. For each node $u = (u_1, \ldots, u_{|u|}) \in \mathcal{T}^+$, let $|u|$ denote its depth.

For a sampled path

$$P = (x_1, \ldots, x_k) \sim \tilde{p},$$

define the length-d prefix

$$P_{\le d} := (x_1, \ldots, x_d).$$

The accepted-prefix length under tree $\mathcal{T}$ is

$$\alpha_{\mathcal{T}}(P) := \max\{d : P_{\le d} \in \mathcal{T}\}.$$

Since $\mathcal{T}$ is prefix-closed, the event $P_{\le d} \in \mathcal{T}$ implies $P_{\le j} \in \mathcal{T}$ for all $j \le d$. Hence,

$$\alpha_{\mathcal{T}}(P) = \sum_{d=1}^{k} \mathbf{1}\{P_{\le d} \in \mathcal{T}\}.$$

Equivalently,

$$\alpha_{\mathcal{T}}(P) = \sum_{u \in \mathcal{T}^+} \mathbf{1}\{P_{\le |u|} = u\}.$$

Taking expectation with respect to $P \sim \tilde{p}$ and applying linearity of expectation,

$$\mathbb{E}_{P\sim\tilde{p}}[\alpha_{\mathcal{T}}(P)] = \sum_{u \in \mathcal{T}^+} \Pr_{P\sim\tilde{p}}(P_{\le |u|} = u).$$

Define the surrogate prefix mass

$$\tilde{p}(u) := \Pr_{P\sim\tilde{p}}(P_{\le |u|} = u).$$

Then

$$\mathbb{E}_{P\sim\tilde{p}}[\alpha_{\mathcal{T}}(P)] = \sum_{u \in \mathcal{T}^+} \tilde{p}(u).$$

If the root node is excluded by convention, then $\mathcal{T}^+ = \mathcal{T}$, yielding Eq. (3).

B. KL-Regularized Derivation of the Prefix-Aligned Surrogate

We consider the variational objective

$$\min_{p\in\Delta(\mathcal{V})} \ \mathrm{KL}(p \,\|\, q_d(\cdot)) - \lambda_d \mathbb{E}_{t\sim p}\big[\log \rho_d(t \mid c_d)\big], \tag{11}$$

where q_d denotes the diffusion drafter marginal, $\rho_d(\cdot \mid c_d)$ is a prefix-conditioned compatibility signal, and $\Delta(\mathcal{V})$ denotes the probability simplex over the vocabulary. We assume $q_d(t) > 0$ and $\rho_d(t \mid c_d) > 0$ on the candidate support.

Expanding the KL divergence,

$$\mathrm{KL}(p\|q_d) = \sum_t p(t) \log \frac{p(t)}{q_d(t)},$$

the objective becomes

$$\mathcal{L}(p) = \sum_t p(t) \log \frac{p(t)}{q_d(t)} - \lambda_d \sum_t p(t) \log \rho_d(t \mid c_d).$$

Including the normalization constraint $\sum_t p(t) = 1$ with Lagrange multiplier μ, we obtain

$$\mathcal{J}(p) = \sum_t p(t) \log \frac{p(t)}{q_d(t)} - \lambda_d \sum_t p(t) \log \rho_d(t \mid c_d) + \mu \left(\sum_t p(t) - 1 \right).$$

Taking derivatives with respect to $p(t)$,

$$\frac{\partial \mathcal{J}}{\partial p(t)} = \log p(t) - \log q_d(t) + 1 - \lambda_d \log \rho_d(t \mid c_d) + \mu.$$

Setting the derivative to zero yields

$$\log p_d^\star(t) = \log q_d(t) + \lambda_d \log \rho_d(t \mid c_d) + C,$$

where C absorbs constants independent of t. Exponentiating both sides,

$$p_d^\star(t) \propto q_d(t)\, \rho_d(t \mid c_d)^{\lambda_d}.$$

Equivalently,

$$p_d^\star(t \mid c_d) = \frac{q_d(t) \rho_d(t \mid c_d)^{\lambda_d}}{Z_d(c_d)}, \qquad Z_d(c_d) = \sum_{t' \in \mathcal{V}} q_d(t') \rho_d(t' \mid c_d)^{\lambda_d}.$$

Thus, the optimal surrogate takes a product-of-experts form, combining the calibrated marginal signal q_d with a prefix-conditioned correction ρ_d.

C. PRESTO on Hybrid dLLMs at Quadratic Self Speculation Decoding

We provide a detailed walk-through of how PRESTO is instantiated on Hybrid dLLMs at quadratic self speculation decoding mode, corresponding to Figure 3. Unlike the standard speculative decoding setup where a small drafter proposes tokens for a larger verifier, self-speculative dLLMs use *the same dLLM* as both drafter and verifier: a single forward pass simultaneously verifies previously drafted tokens and speculates new ones at trailing mask positions. PRESTO turns this single-model pipeline into a tree-based drafter without any additional model.

At the start of round r, the input sequence consists of three contiguous regions:

- a **committed prefix** of accepted tokens from previous rounds (green tokens with ✓ in Figure 3, e.g. `C, D`);
- a **verification region** containing the draft tokens proposed in round $r-1$ (e.g. `E, F, G`), some of which will be accepted and some rejected;
- a **speculation region** of mask tokens (`M`) appended to the right, at which the dLLM will produce new draft candidates.

A single forward pass over this composite input yields outputs that serve both roles below.

(I) AR verification of the previous draft tree. On the verification region, PRESTO uses the exact KV cache of the committed prefix together with an *AR tree attention mask* that restricts each draft token to attend only to its ancestors in the previous round's draft tree. This lets a single forward verify all paths of the tree in parallel: each path is checked left-to-right against the model's predictions, and the longest accepted prefix across paths is committed (`D`→`F` in Figure 3, where `E` on the sibling path and `G` on the same path are rejected). The position immediately after the last accepted token is filled with the model's own prediction (`F'`, `G'`, `H'` in Figure 3), and the accepted tokens become part of the committed prefix for round $r+1$.

(II) Diffusion-based speculation along the accepted path. The speculation region is structured as a set of mask slots attached to each draft token from the previous round, mirroring the previous round's draft tree (Figure 3 shows two candidate paths under verification, `D`→`E` and `D`→`F`→`G`). The dLLM denoises all of these mask positions in parallel within the same forward, but only the mask slots hanging off the *accepted* path are used for the next round's drafting. In Figure 3, `F` is accepted and `G` is rejected, so PRESTO reads top-b candidates from the mask slots following `F` ($b = 2$ in the figure, e.g. $\{H_1, H_2\}$ at the first such position, $\{I_1, I_2\}$ at the next, and so on). The mask slots attached to rejected branches are produced by the same forward but discarded (marked "Unused" in Figure 3), since their predictions were conditioned on a context that is no longer part of the committed prefix.

(III) Draft tree construction. The per-position top-b candidates are assembled into a draft tree following the priority-based expansion of Section 4.2. The root corresponds to the first speculation position, and at depth d each node has up to b children drawn from the top-b candidates at depth $d+1$. Each child inherits its parent's cumulative path score S and adds the local token score $s_{d+1,t}(c)$, so scores are computed incrementally with no redundant work. The frontier is expanded under beam search with global retention until the tree contains B nodes, after which the top-B paths under S are kept (Algorithm 1). Crucially, the candidate logits at every depth come from the *same* forward pass that performed verification, so tree construction itself adds no additional dLLM forwards.

(IV) Flattening for the next forward pass. The draft tree is then linearised into the input sequence for round $r+1$ (bottom of Figure 3). Each draft token is followed by a block of mask tokens that will host the next round's speculation, while the tree's parent–child structure is encoded in the AR tree attention mask used during verification. The result is a single input on which the next forward pass again performs verification and speculation jointly, completing the cycle.

D. End-to-End Throughput Results for dFlash

Table 5 | Average absolute decoding throughput (tokens/s) on Qwen3 models with thinking mode disabled. Speedup is measured against autoregressive decoding on torch implementation with single batch size.

Model	Temp.	dFlash	PRESTO	Gain
Qwen3-4B	$T = 0$	211 (4.9×)	**314** (7.3×)	1.49×
	$T = 1$	168 (3.9×)	**258** (6.0×)	1.54×
Qwen3-8B	$T = 0$	173 (4.8×)	**263** (7.3×)	1.52×
	$T = 1$	137 (3.8×)	**216** (6.0×)	1.58×

E. End-to-End Throughput Results for Nemotron-Labs-Diffusion

Table 6 | Absolute decoding throughput (tokens/s), average acceptance length (τ), and average throughput improvement on Nemotron-Labs-Diffusion-8B under linear self-speculative decoding with a maximum generation length of 2048 tokens based on SGLang implementation.

Model	Method	Math								Code								Chat				*Avg.*	
		GSM8K		MATH-500		AIME24		AIME25		HumanEval		MBPP		LCB		SWE-Bench		MT-Bench		Alpaca		*Avg.*	
	Temperature = 0	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ
8B	AR	151	1.0	139	1.0	130	1.0	127	1.0	142	1.0	148	1.0	126	1.0	125	1.0	142	1.0	145	1.0	136	1.0
	NLD	868	10.8	901	12.2	746	11.0	700	10.5	710	9.3	659	8.2	593	9.0	471	6.8	402	5.2	363	4.6	641	8.7
	NLD w/ PRESTO	908	11.9	933	13.2	792	12.2	731	11.5	738	10.3	681	9.1	623	10.0	506	7.9	460	6.5	420	5.8	679	9.9
	× **Gain**	**1.05×**		**1.03×**		**1.06×**		**1.04×**		**1.04×**		**1.03×**		**1.05×**		**1.07×**		**1.14×**		**1.16×**		**1.06×**	
	Temperature = 1	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ	Thr.	τ
8B	AR	150.8	1.0	138.4	1.0	128.8	1.0	125.4	1.0	141.5	1.0	147.1	1.0	125.4	1.0	124.2	1.0	141.5	1.0	144.5	1.0	136.7	1.0
	NLD	431.6	5.6	401.1	5.7	315.9	5.0	330.0	4.9	322.0	4.6	310.4	4.2	250.6	4.0	175.8	2.9	194.2	2.7	176.0	2.3	290.7	4.2
	NLD w/ PRESTO	470.6	6.1	452.6	7.1	378.2	5.7	357.7	6.4	376.1	5.5	376.0	5.0	284.3	4.8	221.6	3.6	252.0	3.5	213.0	3.2	338.2	5.1
	× **Gain**	**1.09×**		**1.13×**		**1.20×**		**1.08×**		**1.17×**		**1.21×**		**1.13×**		**1.26×**		**1.30×**		**1.21×**		**1.16×**	

F. Entropy-adaptive λ_d

In addition to a fixed interpolation coefficient λ_d, we explored an entropy-adaptive variant that scales the prefix correction by the uncertainty of the diffusion drafter. Let

$$H_d = -\sum_{w \in \mathcal{V}} q_d(w) \log q_d(w)$$

denote the entropy of the diffusion marginal at depth d. We define

$$\lambda_d(H_d) = \beta \cdot \frac{H_d}{H_d + C},$$

where $\beta > 0$ controls the maximum correction strength and $C > 0$ is a stabilising constant, and substitute this into the token-level score from Section 4:

$$s_{d,t}(c_d) = \log q_d(t) + \lambda_d(H_d)\ \log \rho_d(t \mid c_d).$$

Under this schedule, low-entropy (high-confidence) positions assign small weight to the prefix correction and rely primarily on the calibrated marginal q_d, while high-entropy positions up-weight the prefix-conditioned signal ρ_d. The fixed-λ_d variant used in our main experiments is recovered by replacing $\lambda_d(H_d)$ with a constant.